

ARCHITECTURE GITHUB DE LA PREUVE DE CONCEPT

Projet 11 - PoC de Validation d'une Architecture Micro-services

Version : 1.0

Clément Hindié

Historique des versions

Date	Auteur	Commentaires
10 août 2026	Clément Hindié	Création du document

Table des matières

1.	Objet du document.....	3
2.	Contexte.....	3
3.	Principes retenus.....	3
4.	Organisation des dépôts.....	4
5.	Organisation du dépôt de code	4
6.	Stratégie de branches	5
7.	Workflow Git.....	5
8.	Justification de l'approche retenue	6
9.	Conclusion.....	6

1. Objet du document

Le présent document a pour objectif de définir l'organisation GitHub retenue pour la preuve de concept.

Cette organisation constitue le premier élément d'industrialisation du projet. Elle permet d'encadrer les activités de développement, de faciliter la collaboration entre les différents composants de la solution et de préparer la mise en œuvre de l'intégration continue.

L'objectif recherché n'est pas de mettre en place une organisation complexe mais de disposer d'un cadre cohérent permettant de démontrer les pratiques de gestion de version et d'industrialisation attendues dans le contexte d'une architecture micro-services.

2. Contexte

L'architecture applicative précédemment définie repose sur cinq microservices indépendants. Ce découpage architectural doit être reflété dans l'organisation du code source afin de conserver une cohérence entre l'architecture logique et l'organisation des travaux de développement.

L'organisation GitHub retenue doit permettre de maintenir l'indépendance des composants tout en facilitant leur évolution et leur intégration progressive dans la preuve de concept.

Cette approche contribue également à démontrer l'application des principes d'intégration continue et de traçabilité définis dans les exigences de la preuve de concept.

3. Principes retenus

L'organisation GitHub s'appuie sur trois principes.

Le premier consiste à séparer les artefacts d'architecture des artefacts logiciels. Cette distinction facilite la lecture des livrables et permet de faire évoluer la documentation indépendamment du code.

Le second consiste à conserver une correspondance directe entre les micro-services identifiés dans l'architecture et les composants développés dans la preuve de concept.

Le troisième consiste à privilégier une organisation simple afin de conserver un périmètre maîtrisable dans le contexte du projet.

4. Organisation des dépôts

L'organisation proposée repose sur deux dépôts principaux.

Le premier dépôt regroupe l'ensemble des documents d'architecture produits au cours du projet. Il constitue la source de référence des décisions architecturales, des contrats REST, des événements métier et des différents livrables associés à la preuve de concept.

Le second dépôt regroupe les composants logiciels nécessaires à l'implémentation de la preuve de concept. Les différents micro-services y sont organisés dans des répertoires dédiés afin de conserver une correspondance claire avec l'architecture applicative.

Dépôt	Contenu	Objectif
OC-Project11-architecture	Documents d'architecture et livrables	Conserver la référence documentaire du projet
OC-Project11-poc	Code source et configuration	Implémenter la preuve de concept

Afficher plus de lignes

Cette organisation permet d'isoler les activités de conception des activités de réalisation tout en conservant une traçabilité complète des livrables produits.

5. Organisation du dépôt de code

Le dépôt de code est structuré autour des micro-services retenus lors des travaux d'architecture.

Répertoire	Responsabilité
routing-service	Calcul des distances et temps de trajet
hospital-service	Gestion du référentiel hospitalier
recommendation-service	Production des recommandations
bed-reservation-service	Gestion des réservations
frontend	Interface de démonstration de la preuve de concept

Cette organisation permet d'identifier immédiatement les responsabilités portées par chaque composant et facilite la mise en place future de chaînes d'intégration indépendantes.

6. Stratégie de branches

La stratégie de branches doit rester compatible avec le périmètre de la preuve de concept tout en démontrant les bonnes pratiques de gestion de version.

Une branche principale représente la version stable de la preuve de concept.

Une branche d'intégration permet de regrouper les travaux avant leur validation.

Les développements sont réalisés dans des branches dédiées associées aux fonctionnalités ou aux composants concernés.

Branche	Objectif
main	Version stable de référence
develop	Intégration des évolutions
feature/*	Développement d'une fonctionnalité

Cette approche permet de démontrer un cycle de développement maîtrisé tout en conservant une organisation simple.

7. Workflow Git

Le workflow retenu repose sur un mécanisme de fusion contrôlée.

Les travaux sont réalisés dans une branche de fonctionnalité puis intégrés dans la branche develop après validation. Les modifications validées sont ensuite intégrées dans la branche main afin de constituer une nouvelle version stable de la preuve de concept.

Étape	Branche source	Branche cible
Développement	feature/*	develop
Validation	develop	main
Publication	main	Version de référence

Cette approche garantit que la branche principale reste en permanence dans un état exploitable.

8. Justification de l'approche retenue

L'organisation proposée répond directement aux contraintes de la preuve de concept.

La séparation entre architecture et implémentation permet d'isoler les décisions de conception des travaux de développement. Le découpage du dépôt de code reflète l'architecture micro-services définie précédemment et facilite l'évolution indépendante des composants.

La stratégie de branches retenue demeure volontairement simple afin de concentrer les efforts sur la validation des principes architecturaux plutôt que sur la gestion d'un processus de développement complexe.

Cette organisation constitue également une base adaptée à la mise en œuvre de GitHub Actions qui représentera l'étape suivante de la démarche d'industrialisation.

9. Conclusion

L'organisation GitHub retenue repose sur deux dépôts distincts, une structure de code alignée sur l'architecture micro-services et une stratégie de branches simplifiée adaptée au contexte de la preuve de concept.

Cette approche permet de démontrer les pratiques de gestion de version attendues dans le cadre du projet tout en préparant la mise en œuvre de l'intégration continue.

Le présent document constitue la base de travail de l'étape suivante consacrée à la définition de la stratégie GitHub Actions et des mécanismes d'intégration continue de la preuve de concept.